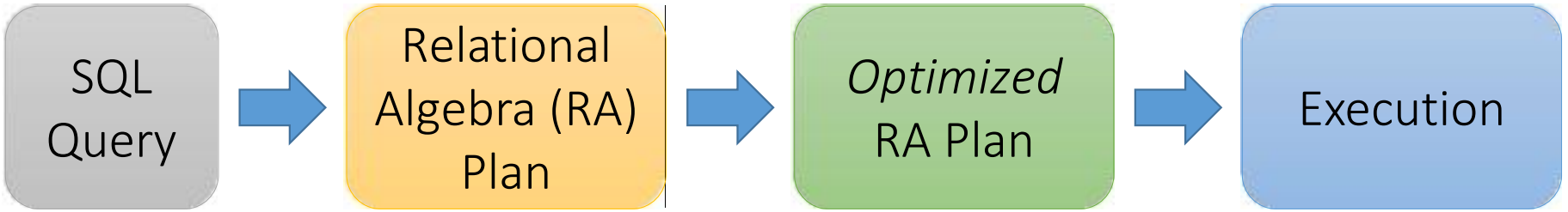
CSE 215: Database

Lecture 11

Chapter 2: The Relational Algebra and Triggers

RDBMS Architecture

How does a SQL engine work ?



Declarative query (from user)

Translate to relational algebra expression

Find logically equivalent- but more efficient- RA expression

Execute each operator of the optimized plan!

RDBMS Architecture

How does a SQL engine work ?

Relational Algebra allows us to translate declarative (SQL) queries into precise and optimizable expressions!

Relational Algebra (RA)

- Five **basic** operators:

1. Selection: σ
2. Projection: Π
3. Cartesian Product: $\times$
4. Union: $\cup$
5. Difference: -

We'll look at these first!

Then we'll look see these set operations

- Derived or auxiliary operators:
  - Intersection
  - Joins (natural,equi-join, theta join)
  - Renaming: ρ

And also at one example of a derived operator (natural join) and a special operator (renaming)

Keep in mind: RA operates on sets!

- RDBMSs use *Bags* (*multisets)*, however in relational algebra formalism we will consider **sets!**

- Also: we will consider the ***named perspective***, where every attribute must have a unique name
  - →attribute order does not matter…

Now on to the basic RA operators…

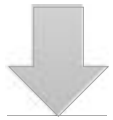
1. Selection (σ)

- Returns all tuples which satisfy a condition
- Notation: $\sigma_c(R)$
- Examples
  - $\sigma_{Salary > 40000}$ (Employee)
  - $\sigma_{name = 'Smith'}$ (Employee)
- The condition c can use =, <, ≤, >, ≥, <>

Students(sid,sname,gpa)

SQL:

```
SELECT *
FROM Students
WHERE gpa > 3.5;
```



RA:

$$\sigma_{gpa>3.5}(Students)$$

Another example:

SSN	Name	Salary
1234545	John	200000
5423341	Smith	600000
4352342	Fred	500000

$\sigma_{Salary > 40000}$ (Employee)

SSN	Name	Salary
5423341	Smith	600000
4352342	Fred	500000

2. Projection (Π)

- Eliminates columns, then removes duplicates
- Notation: $\Pi_{A1,\ldots,An}(R)$
- Example: project social-security number and names:
  - $\Pi_{SSN,\ Name}$ (Employee)
  - Output schema: Answer(SSN, Name)

Students(sid,sname,gpa)

SQL:

```
SELECT DISTINCT
  sname,
  gpa
FROM Students;
```

RA:

$$\Pi_{sname,gpa}(Students)$$

Another example:

SSN	Name	Salary
1234545	John	200000
5423341	John	600000
4352342	John	200000

$\Pi_{Name,Salary}$ (Employee)

Name	Salary
John	200000
John	600000

Note that RA Operators are Compositional!

Students(sid,sname,gpa)

```
SELECT DISTINCT
  sname,
  gpa
FROM Students
WHERE gpa > 3.5;
```

$$\Pi_{sname,gpa}(\sigma_{gpa>3.5}(Students))$$

$$\sigma_{gpa>3.5}(\Pi_{sname,gpa}(Students))$$

How do we represent this query in RA?

Are these logically equivalent?

Note that RA Operators are Compositional!

Students(sid,sname,gpa)

```
SELECT DISTINCT
  sid,
  sname
FROM Students
WHERE gpa > 3.5;
```

How do we represent this query in RA?

$$\Pi_{sid,sname}(\sigma_{gpa>3.5}(Students))$$

$$\sigma_{gpa>3.5}(\Pi_{sid,sname}(Students))$$

Are these logically equivalent?

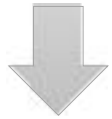
3. Cross-Product (×)

- Each tuple in R1 with each tuple in R2
- Notation: R1 × R2
- Example:
  - Employee × Dependents
- Rare in practice; mainly used to express joins

Students(sid,sname,gpa)
People(ssn,pname,address)

SQL:

```
SELECT *
FROM Students, People;
```



RA:

$$Students \times People$$

Another example:

People

ssn	pname	address
1234545	John	216 Rosse
5423341	Bob	217 Rosse

$\times$

Students

sid	sname	gpa
001	John	3.4
002	Bob	1.3

$Students \times People$

ssn	pname	address	sid	sname	gpa
1234545	John	216 Rosse	001	John	3.4
5423341	Bob	217 Rosse	001	John	3.4
1234545	John	216 Rosse	002	Bob	1.3
5423341	Bob	216 Rosse	002	Bob	1.3

Renaming:

- Changes the schema, not the instance
- A 'special' operator- neither basic nor derived
- Notation: $\rho_{S(B1,\ldots,Bn)}(R)$

- **Note: this is shorthand for the proper form (since names, not order matters!):**
  - $\rho_{S(A1\rightarrow B1,\ldots,An\rightarrow Bn)}(R)$

Renaming:

Students(sid,sname,gpa)

- Changes the schema, not the instance
- A 'special' operator- neither basic nor derived
- Notation: $\rho_{S(B1,\ldots,Bn)}(R)$

- **Note: this is shorthand for the proper form (since names, not order matters!):**
  - $\rho_{S(A1\rightarrow B1,\ldots,An\rightarrow Bn)}(R)$

SQL:

```
SELECT
  sid AS studId,
  sname AS name,
  gpa AS gradePtAvg
FROM Students;
```

RA:

$$\rho_{Students(studId,name,gradePtAvg)}(Students)$$

Another example:

Students

sid	sname	gpa
001	John	3.4
002	Bob	1.3

$$\rho_{Students(studId,name,gradePtAvg)}(Students)$$

Students

studId	name	gradePtAvg
001	John	3.4
002	Bob	1.3

Natural Join (⋈)

- Notation: $R_1 \bowtie R_2$
- Joins R_1 and R_2 on *equality of all shared attributes*
  - If R_1 has attribute set A, and R_2 has attribute set B, and they share attributes $A \cap B = C$, can also be written: $R_1 \bowtie_C R_2$
- Our first example of a *derived* RAoperator:
  - Meaning: $R_1 \bowtie R_2 = \Pi_{A \cup B}(\sigma_{C=D}(\rho_{C \to D}(R_1) \times R_2))$
  - Where:
    - The rename $\rho_{C \to D}$ renames the shared attributes in one of the relations
    - The selection $\sigma_{C=D}$ checks equality of the shared attributes
    - The projection $\Pi_{A \cup B}$ eliminates the duplicate common attributes

Students(sid,name,gpa)
People(ssn,name,address)

SQL:

```
SELECT DISTINCT
  ssid, S.name, gpa,
  ssn, address
FROM
  Students S,
  People P
WHERE S.name = P.name;
```

RA:

$$Students \bowtie People$$

Another example:

Students S

sid	S.name	gpa
001	John	3.4
002	Bob	1.3

$\bowtie$

People P

ssn	P.name	address
1234545	John	216 Rosse
5423341	Bob	217 Rosse

$$Students \bowtie People$$

sid	S.name	gpa	ssn	address
001	John	3.4	1234545	216 Rosse
002	Bob	1.3	5423341	216 Rosse

Natural Join

- Given schemas R(A, B, C, D), S(A, C, E), what is the schema of R ⋈ S ?
- Given R(A, B, C), S(D, E), what is R ⋈ S ?
- Given R(A, B), S(A, B), what is R ⋈ S ?

Example: Converting SFW Query -> RA

Students(sid,sname,gpa)
People(ssn,sname,address)

```
SELECT DISTINCT
  gpa,
  address
FROM Students S,
     People P
WHERE gpa > 3.5 AND
   S.sname = P.sname;
```

$$\Pi_{gpa,address}(\sigma_{gpa>3.5}(S \bowtie P))$$

How do we represent this query in RA?

Advanced Relational Algebra

1. Set Operations in RA
2. Fancier RA
3. Extensions & Limitations

Relational Algebra (RA)

- Five **basic** operators:
  1. Selection: σ
  2. Projection: Π
  3. Cartesian Product: $\times$
  4. Union: $\cup$
  5. Difference: -

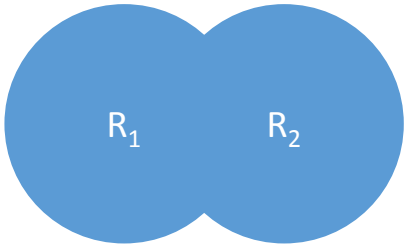
We'll look at these

- Derived or auxiliary operators:
  - Intersection
  - Joins (natural,equi-join, theta join, semi-join)
  - Renaming: ρ

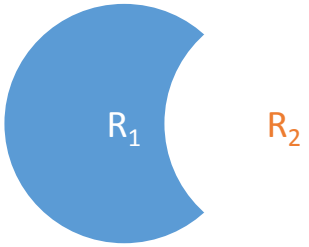
And also at some of these derived operators

1. Union (∪) and 2. Difference (–)

- R1 ∪ R2
- Example:
  - ActiveEmployees ∪ RetiredEmployees

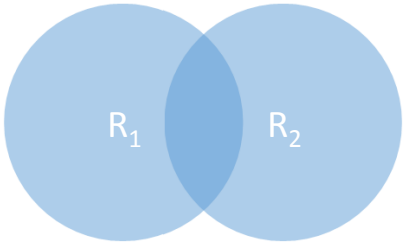


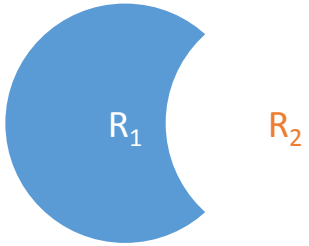
- R1 – R2
- Example:
  - AllEmployees - RetiredEmployees



What about Intersection (∩) ?

- It is a derived operator
- R1 ∩ R2 = ?
- R1 ∩ R2 = R1 – (R1 – R2)
- Example
  - UnionizedEmployees ∩ RetiredEmployees





R1 – R2

Fancier RA

Theta Join ($\bowtie_\theta$)

- A join that involves a predicate
- R1 $\bowtie_\theta$ R2 = σ_θ (R1 $\times$ R2)
- Here θ can be any condition

Note that natural join is a theta join + a projection.

Students(sid,sname,gpa)
People(ssn,pname,address)

SQL:

```
SELECT *
FROM
  Students,People
WHERE θ;
```

RA:

$$Students \bowtie_\theta People$$

Equi-join ($\bowtie_{A=B}$)

- A theta join where θ is an equality
- $R1 \bowtie_{A=B} R2 = \sigma_{A=B} (R1 \times R2)$
- Example:
  - Employee $\bowtie_{SSN=SSN}$ Dependents

Most common join in practice!

Students(sid,sname,gpa)
People(ssn,pname,address)

SQL:

```
SELECT *
FROM
  Students S,
  People P
WHERE sname = pname;
```

RA:

$$S \bowtie_{sname=pname} P$$

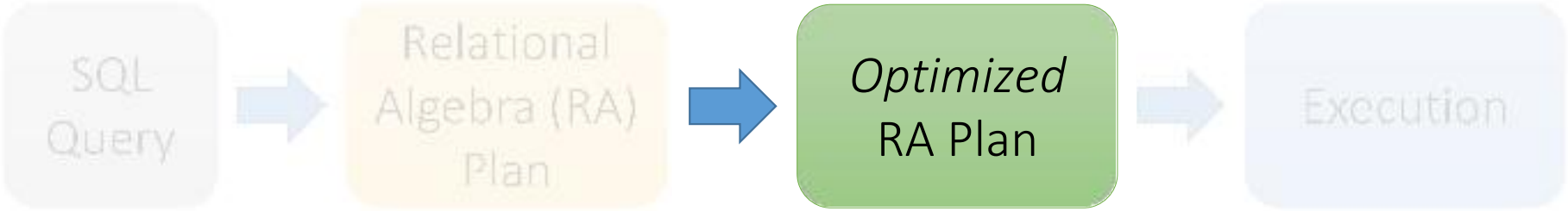
Optimization

Logical Equivalence of RA Plans

- Given relations R(A,B) and S(B,C):
  - Here, projection & selection commute:
    - $\sigma_{A=5}(\Pi_A(R)) = \Pi_A(\sigma_{A=5}(R))$
  - What about here?
    - $\sigma_{A=5}(\Pi_B(R)) \stackrel{?}{=} \Pi_B(\sigma_{A=5}(R))$

RDBMS Architecture

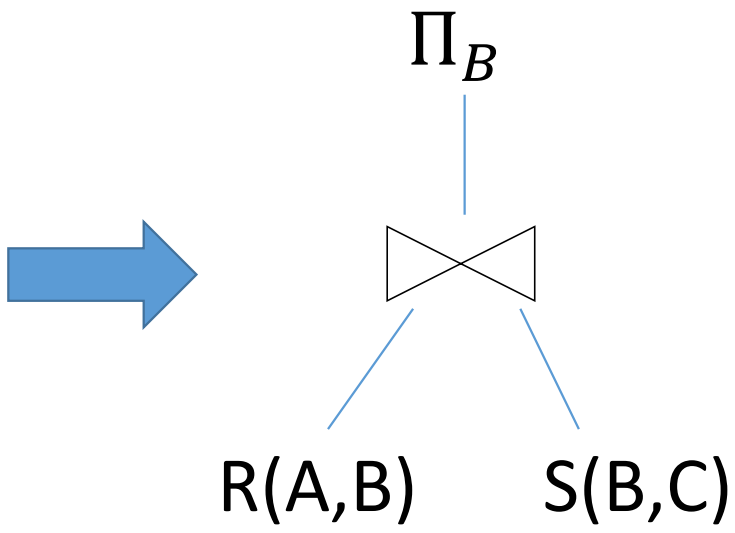
How does a SQL engine work ?



We'll get a flavor of how to optimize on these plans now

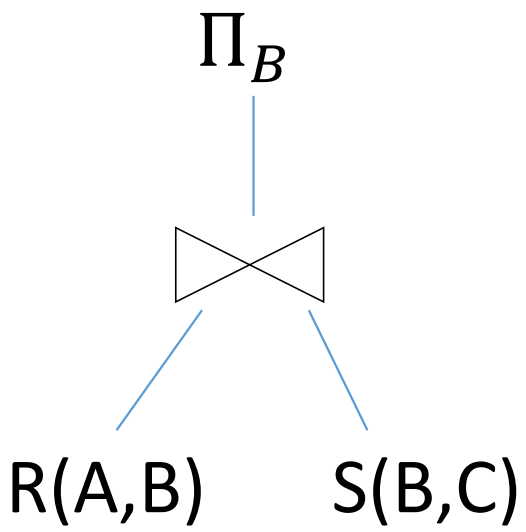
Note: We can visualize the plan as a tree

$$\Pi_B(R(A,B) \bowtie S(B,C))$$



Bottom-up tree traversal = order of operation execution!

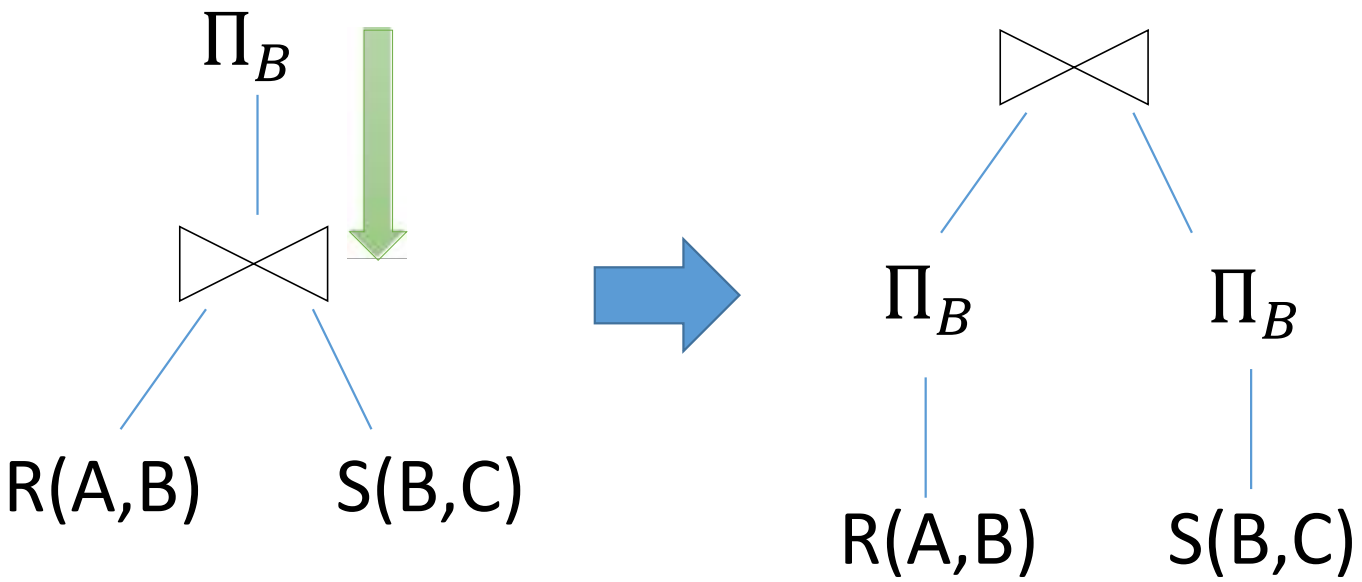
A simple plan



What SQL query does this correspond to?

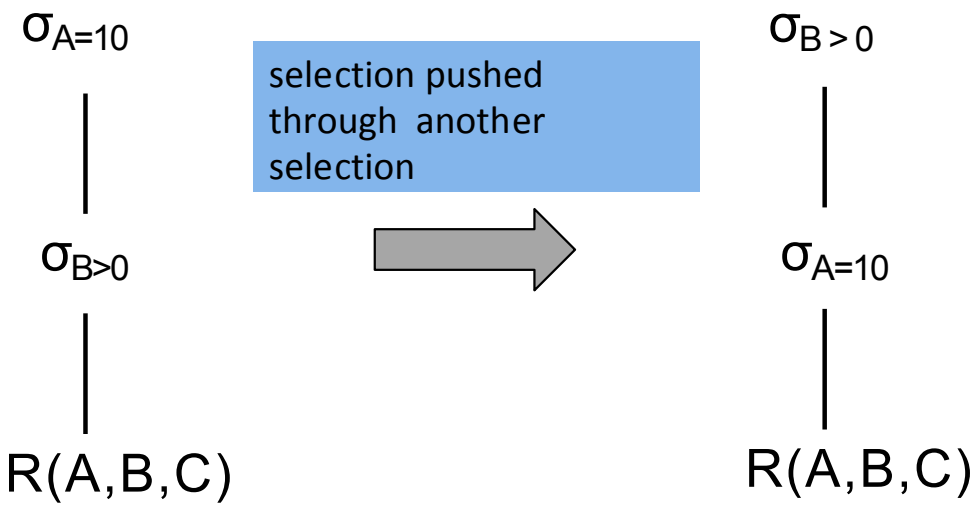
Are there any logically equivalent RA expressions?

"Pushing down" projection



Why might we prefer this plan?

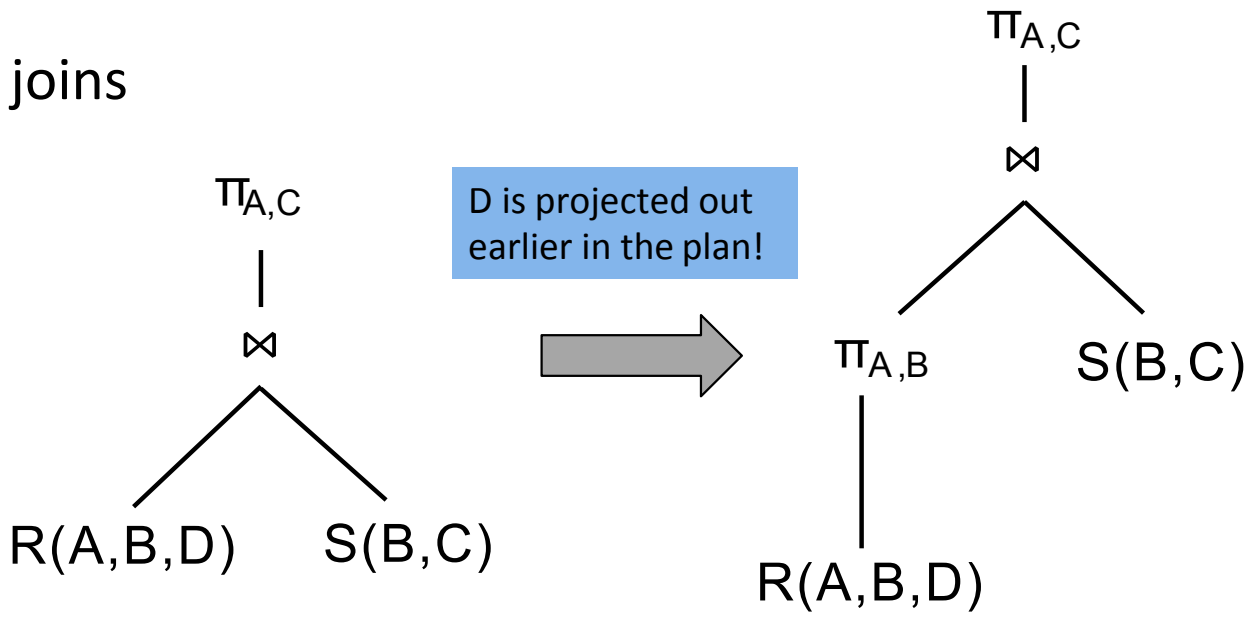
It is always possible to change the order of selections



PUSHING DOWN PROJECTIONS

A projection can be pushed down through

- joins



Takeaways

- This process is called logical optimization
- Many equivalent plans used to search for “good plans”
- Relational algebra is an important abstraction.

RA commutators

- The basic commutators:
  - Push **projection** through **(1) selection**, **(2) join**
  - Push **selection** through **(3) selection, (4) projection, (5) join**
  - *Also:* Joins can be re-ordered!

This simple set of tools allows us to greatly improve the execution time of queries by optimizing RA plans!

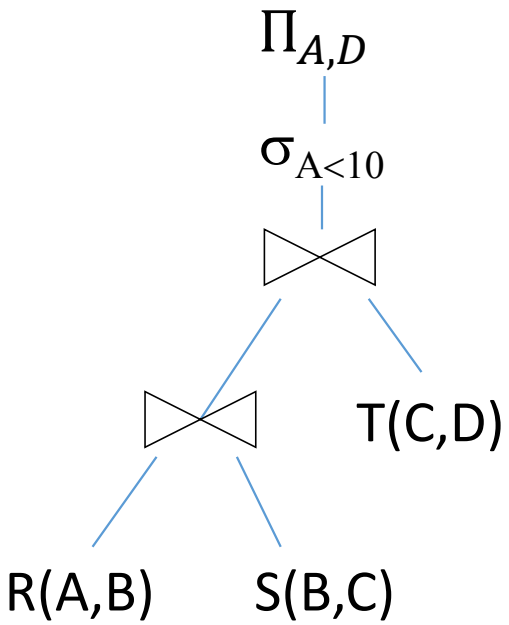
Optimizing the SFW RA Plan

Translating to RA

R(A,B) S(B,C) T(C,D)

```
SELECT R.A, T.D
FROM R,S,T
WHERE R.B = S.B
  AND S.C = T.C
  AND R.A < 10;
```

$$\Pi_{A,D}(\sigma_{A<10}(T \bowtie (R \bowtie S)))$$



Logical Optimization

- Heuristically, we want selections and projections to occur as early as possible in the plan
  - Terminology: “push down **selections**” and “pushing down **projections.”**

- **Intuition:** We will have fewer tuples in a plan.

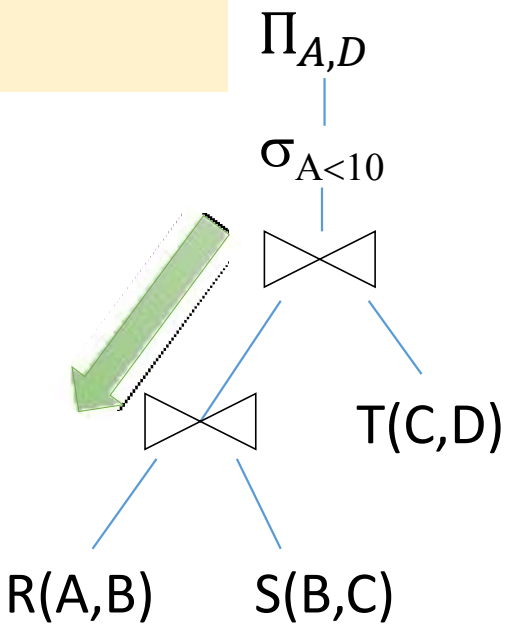
Optimizing RA Plan

R(A,B) S(B,C) T(C,D)

```
SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
  AND S.C = T.C
  AND R.A < 10;
```

Push down selection on A so it occurs earlier

$$\Pi_{A,D}(\sigma_{A<10}(T \bowtie (R \bowtie S)))$$



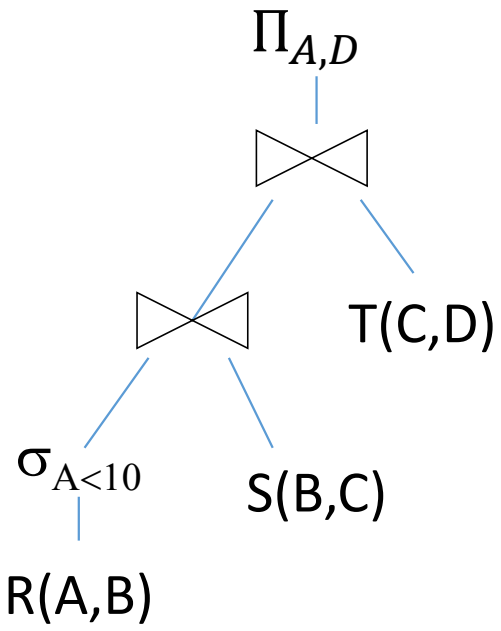
Optimizing RA Plan

R(A,B) S(B,C) T(C,D)

```
SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
  AND S.C = T.C
  AND R.A < 10;
```

Push down selection on A so it occurs earlier

$$\Pi_{A,D}\left(T \bowtie \left(\sigma_{A<10}(R) \bowtie S\right)\right)$$



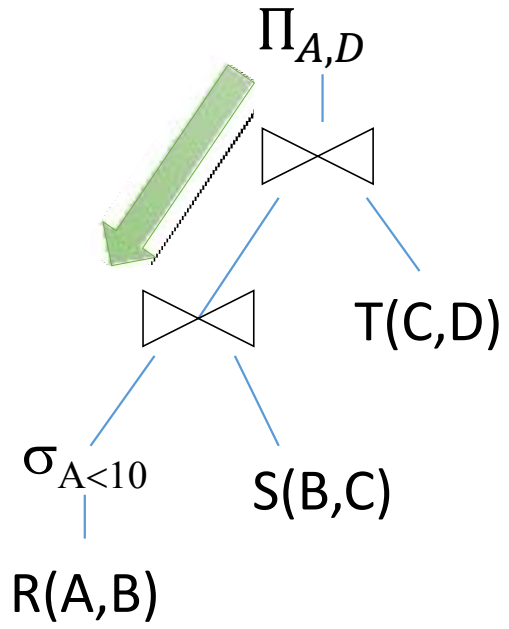
Optimizing RA Plan

R(A,B) S(B,C) T(C,D)

```
SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
  AND S.C = T.C
  AND R.A < 10;
```

Push down projection so it occurs earlier

$$\Pi_{A,D}\left(T \bowtie \left(\sigma_{A<10}(R) \bowtie S\right)\right)$$



Optimizing RA Plan

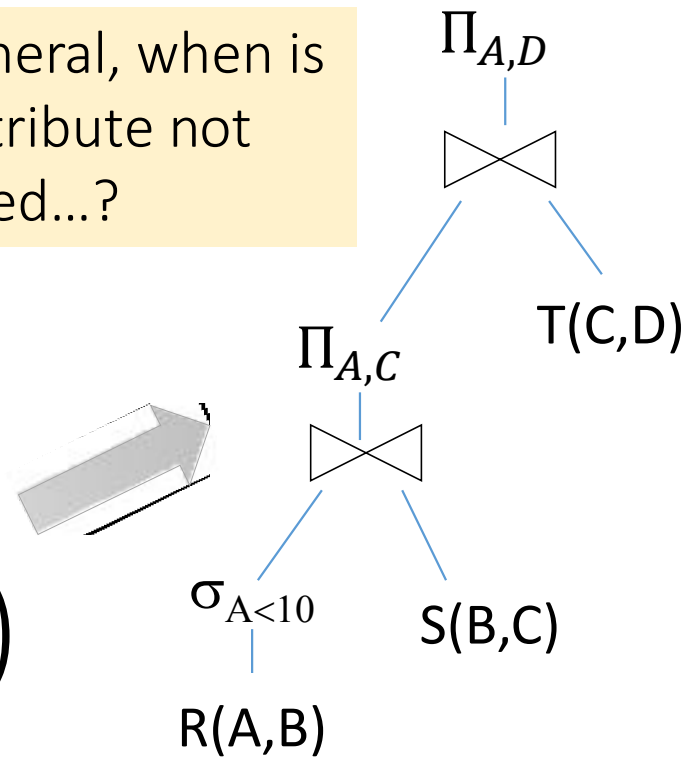
R(A,B) S(B,C) T(C,D)

```
SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
  AND S.C = T.C
  AND R.A < 10;
```

We eliminate B earlier!

In general, when is an attribute not needed…?

$$\Pi_{A,D}\left(T \bowtie \Pi_{A,c}(\sigma_{A<10}(R) \bowtie S)\right)$$



Example (PPLP)

- What PC models have a speed of at least 3.00?

$$\text{Answer(model)} := \Pi_{\text{model}}(\sigma_{\text{speed} >= 3.00}(\text{PC}))$$

- Which manufacturers make laptops with a hard disk of at least 100GB.

$$\text{Answer(maker)} := \Pi_{\text{maker}}(\sigma_{\text{hd} >= 100}(\text{Product} \bowtie \text{Laptop}))$$

Example (PPLP)

- Find the model numbers of all color laser printers?

$$\Pi_{model}(\sigma_{color='T' \text{ AND } type = 'laser'}(Printer))$$

- Find those manufacturer that sells Laptops but not PC's.

$$\Pi_{maker}(\sigma_{type='laptop'}(Product)) - \Pi_{maker}(\sigma_{type='pc'}(Product))$$

Example (PPLP)

- Find those hard disk sizes that occur two or more PC's?

$$\Pi_{pc1.hd}(\sigma_{pc1.model <> pc2.model \text{ AND } pc1.hd = pc2.hd}(\rho_{pc1}(PC) \times \rho_{pc2}(PC)))$$

- Find those pairs of PC models that have both the same speed and RAM. A pair should be listed only once.

$$\Pi_{pc1.model,pc2.model}(\sigma_{pc1.model < pc2.model \text{ AND } pc1.speed = pc2.speed \text{ AND } pc1.ram = pc2.ram}(\rho_{pc1}(PC) \times \rho_{pc2}(PC)))$$

Example (PPLP)

- Find the manufacturer(s) of the PC or Laptop with highest available speed?

$$R1(\text{model, speed}) := \Pi_{\text{model,speed}}(\text{PC}) \cup \Pi_{\text{model,speed}}(\text{Laptop})$$

$$T(\text{model, speed}) := R1 - \sigma_{R1.\text{speed} < R2.\text{speed}}(R1 \times \rho_{R2}(R1))$$

$$\text{Answer(maker)} := \Pi_{\text{maker}}(T \bowtie \text{Product})$$

- Precedence of relational operators:
  1. [σ, π, ρ] (highest).
  2. [x, ⋈].
  3. ∩.
  4. [∪, —]

Triggers

```
CREATE [OR REPLACE ] TRIGGER trigger_name
{BEFORE | AFTER}
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW]
WHEN (condition)
BEGIN
  Executable-statements
END;
```

INSERT/UPDATE Triggers example

```
CREATE OR REPLACE TRIGGER NoNegative
BEFORE UPDATE OF Price,speed ON PC
REFERENCING new as newTupple
FOR EACH ROW
WHEN (newTupple.price < 0 OR newTupple.speed < 0)
BEGIN
    RAISE_APPLICATION_ERROR(-20000, 'no negative value
allowed');
END;
/
```

Delete Triggers example

```
CREATE OR REPLACE TRIGGER cascading
BEFORE delete ON PC
REFERENCING OLD AS oldTupple
FOR EACH ROW
BEGIN
DELETE FROM Product
WHERE model = :oldTupple.model;
END;
/
```